

ALA Final Exam

Cell-Σ Commissioning — Solution Guide

COURSE	CTS4321C Advanced Linux Administration
TYPE	Practical final exam (real container)
FLAGS	9 (one per commissioning stage; stage 9 is the capstone)
DIFFICULTY	Hard
POINTS	Variable (8 stages + final audit)

CONFIDENTIAL — Instructor solution guide. Not for distribution to students except at the instructor's discretion. Contains flag answers and full walkthrough steps.

Objective

The student is handed a bare, "dark" Sector 7 grid cell (`cell-sigma` , 10.0.1.42) and a commissioning order. They must build it from a minimal base into a fully-operational production grid node, re-performing every skill from the course. Each stage is verified against the cell's **real live state** by the grid (lab-manager) and reveals a flag; the final audit re-checks the entire cell.

This is a real Linux container, not a simulation. The exam services (BIND9, the build toolchain, AIDE, cron, SSH server, iptables) are **not** preinstalled — the student installs and configures everything. The operator account has passwordless `sudo` .

Starting State

- Connect surface: a web terminal (ttyd) as the `operator` account.
- The grid tooling: `commission-check <stage>` (the grader) and a real `systemctl` shim (the cell has no systemd).
- In-cell brief: `cat /opt/grid/commission-brief.txt` .
- The `gridmon` source tree ships at `/opt/grid/source/gridmon/` .
- Planted at boot (for later stages): two rogue processes (ghosthunt) and a prior-intrusion artifact set (coldcase).

Check any stage at any time with `commission-check <stage>` . The list and progress:
`commission-check` .

Step-by-Step Walkthrough

Every command below is copy-paste-safe. Run them in order. Each stage ends by running its `commission-check` , which prints `FLAG: FLAG{...}` once the grid confirms the real state. Submit each flag on the exam page.

Stage 1 — First Light (users + key-only SSH) → flag `stage1_firstlight`

Create the grid-ops service account, install + harden SSH to key-only, install an authorized key, and start sshd.

```

sudo useradd -m -s /bin/bash gridops
sudo apt-get update -qq && sudo apt-get install -y openssh-server
sudo tee /etc/ssh/sshd_config.d/99-grid.conf >/dev/null <<'EOF'
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
EOF
ssh-keygen -t ed25519 -f /tmp/gridops_key -N ""
sudo mkdir -p /home/gridops/.ssh
sudo tee /home/gridops/.ssh/authorized_keys < /tmp/gridops_key.pub
sudo chown -R gridops:gridops /home/gridops/.ssh
sudo chmod 700 /home/gridops/.ssh
sudo chmod 600 /home/gridops/.ssh/authorized_keys
sudo systemctl start ssh
commission-check firstlight

```

Expected:

```

[PASS] service account gridops exists
[PASS] sshd is running
[PASS] password auth disabled
[PASS] pubkey auth enabled
[PASS] root login disabled
[PASS] gridops has a valid authorized key

FLAG: FLAG{cell-sigma_stage1_first_light_harden}

```

Stage 2 — Name Authority (BIND9 DNS) → flag `stage2_dns`

Stand up BIND9 so the grid resolves `cell-sigma` forward and reverse.

```

sudo apt-get install -y bind9 bind9-utils
sudo tee /etc/bind/named.conf.local >/dev/null <<'EOF'
zone "sector7.matrix.net" { type master; file "/etc/bind/db.sector7"; };
zone "1.0.10.in-addr.arpa" { type master; file "/etc/bind/db.10.0.1"; };
EOF
sudo tee /etc/bind/db.sector7 >/dev/null <<'EOF'
$TTL 300
@   IN SOA ns1.sector7.matrix.net. admin.sector7.matrix.net. ( 1 3600 1800 604800 300 )
@   IN NS  ns1.sector7.matrix.net.

```

```

ns1      IN A 10.0.1.1
cell-sigma IN A 10.0.1.42
EOF
sudo tee /etc/bind/db.10.0.1 >/dev/null <<'EOF'
$TTL 300
@   IN SOA ns1.sector7.matrix.net. admin.sector7.matrix.net. ( 1 3600 1800 604800 300 )
@   IN NS  ns1.sector7.matrix.net.
42  IN PTR cell-sigma.sector7.matrix.net.
EOF
sudo systemctl start named
commission-check dns

```

Expected: 3× **[PASS]** then **FLAG: FLAG{cell-sigma_stage2_name_authority_online}**.

Stage 3 — Field Assembly (compile gridmon from source) → flag **stage3_assembly**

No package exists for **gridmon**; compile it from the shipped source.

```

sudo apt-get install -y build-essential
cd /opt/grid/source/gridmon
make
sudo make install
commission-check assembly

```

Expected: 4× **[PASS]** then **FLAG: FLAG{cell-sigma_stage3_field_assembly_complete}**.

Stage 4 — The Watch (bash + cron maintenance) → flag **stage4_watch**

Write a maintenance script, schedule it with cron, and start the cron service.

```

sudo tee /usr/local/sbin/grid-maintenance >/dev/null <<'EOF'
#!/bin/bash
mkdir -p /var/log/grid
echo "$(date -u +%Y-%m-%dT%H:%M:%SZ) cell-sigma heartbeat OK" >> /var/log/grid/
heartbeat.log
EOF
sudo chmod +x /usr/local/sbin/grid-maintenance
sudo apt-get install -y cron
sudo systemctl start cron

```

```
echo "* * * * * /usr/local/sbin/grid-maintenance" | sudo crontab -
commission-check watch
```

Expected: 4× **[PASS]** then **FLAG: FLAG{cell-sigma_stage4_watch_scheduled}**.

Stage 5 — Integrity Seal (AIDE baseline) → flag **stage5_integrity**

Install AIDE and initialize a baseline (this hashes the filesystem — ~20-30s). The **DEBIAN_FRONTEND=noninteractive** prefix is required: AIDE recommends a mail-transport-agent (Postfix), whose installer otherwise opens an interactive "mail server configuration type" prompt that **blocks the terminal**. **noninteractive** makes Postfix install silently with a default (no prompt) while still pulling everything **aideinit** needs.

```
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y aide aide-common
sudo aideinit -y -f
sudo cp -f /var/lib/aide/aide.db.new /var/lib/aide/aide.db
commission-check integrity
```

Expected: 3× **[PASS]** then **FLAG: FLAG{cell-sigma_stage5_integrity_sealed}**.

Stage 6 — Perimeter (firewall policy) → flag **stage6_perimeter**

Default-deny **INPUT**, allowing only loopback, established, SSH (22), DNS (53). **Add the allow rules before setting the DROP policy.**

```
sudo apt-get install -y iptables
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 53 -j ACCEPT
sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT
sudo iptables -P INPUT DROP
commission-check perimeter
```

Expected: 5× **[PASS]** then **FLAG: FLAG{cell-sigma_stage6_perimeter_secured}**.

Stage 7 — Ghost Hunt (clear planted faults) → flag **stage7_ghosthunt**

Two processes masquerade as grid daemons but run from a hidden **/tmp** dir. Find and kill both.

```
ps -eo pid,user,args | grep -E '/tmp/.cache/(grid-health|gridcache)' | grep -v grep
sudo pkill -f /tmp/.cache/grid-health
sudo pkill -f /tmp/.cache/gridcache
commission-check ghosthunt
```

Expected: 2× **[PASS]** then **FLAG: FLAG{cell-sigma_stage7_ghosts_cleared}**.

Stage 8 — Cold Case (forensic remediation) → flag **stage8_coldcase**

A prior intruder left a backdoor account, its sudo grant, and a persistence cron. Investigate (start with the incident report) and remove all three.

```
cat /var/log/grid/intrusion-report.txt
getent passwd gridsvc
sudo ls -la /etc/sudoers.d/ /etc/cron.d/
sudo userdel gridsvc
sudo rm -f /etc/sudoers.d/90-gridsvc /etc/cron.d/grid-telemetry /tmp/.sys/beacon
commission-check coldcase
```

Expected: 3× **[PASS]** then **FLAG: FLAG{cell-sigma_stage8_case_closed}**.

Stage 9 — Commissioning Audit (capstone) → flag **stage9_audit**

The final audit re-checks **every** stage. With all eight complete, run:

```
commission-check audit
```

Expected: 8× **[PASS]** , then:

```
All systems green – requesting final commissioning...

FLAG: FLAG{cell-sigma_commissioned_sector7_node_online}
```

Cell-Σ is commissioned.

Grading Notes (instructor)

- Each flag is held **server-side only** (in lab-manager); it is never present in the container. The grader (`commission-check`) calls the grid, which **independently** `docker exec` s into the live cell to verify real state before returning the flag. A student cannot obtain a flag without the work actually being done on a real cell.
- **Flag sharing is allowed by design** — it supports collective/team operation. Because the integrity is enforced at verification time (real state on a real cell), sharing never becomes credit for zero work.
- The capstone (`audit`) re-runs all eight stage verifiers; it passes only when the whole cell holds at once.
- Architecture + build detail: `_docs/operations/cell-sigma-final-exam-spec-2026-06-24.md` .